

EdgeOne Pages Platform Guide

A Technical Reference Document

Version 1.0 - 2025

1. What is EdgeOne Pages

EdgeOne Pages is a modern edge computing platform designed for deploying full-stack web applications. It combines the speed of a global CDN with the flexibility of serverless functions, allowing developers to build and deploy applications that run close to their users worldwide.

The platform supports static site hosting, server-side rendering (SSR), and API endpoints through its Pages Functions feature. Applications deployed on EdgeOne Pages benefit from automatic HTTPS, global edge caching, and sub-millisecond cold start times.

EdgeOne Pages is particularly well-suited for AI-powered applications, real-time collaboration tools, and content-heavy websites that require both fast static delivery and dynamic server-side processing.

2. Pages Functions Architecture

Pages Functions are serverless functions that run at the edge, close to your users. They are defined by placing JavaScript or TypeScript files in the `/agents` directory of your project. Each file automatically maps to an API route based on its filename.

For example, a file at `agents/chat.ts` becomes accessible at the `/chat` API endpoint. The function receives a request context object that provides access to the request body, headers, query parameters, and a persistent key-value store.

The execution model is event-driven: each incoming HTTP request triggers a function invocation. Functions can return standard `Response` objects, stream data using Server-Sent Events (SSE), or return JSON payloads directly.

Key architectural features include: (1) automatic route mapping from file structure, (2) built-in KV storage via `context.store` for session persistence, (3) environment variable injection for secrets management, and (4) support for both synchronous and streaming responses.

3. The context.store API

Every Pages Function receives a context object that includes a persistent key-value store accessible via `context.store`. This store provides a simple but powerful interface for maintaining state across requests.

The store supports the following operations: `context.store.get(key)` retrieves a value by key, returning null if not found. `context.store.put(key, value)` writes a value (string or `ArrayBuffer`) with an optional expiration TTL in seconds. `context.store.delete(key)` removes a key-value pair.

Common use cases for `context.store` include: storing conversation history for chatbots, caching API responses to reduce latency, maintaining user session data, and tracking rate limits per client IP.

The store is globally distributed and eventually consistent, with writes becoming visible across all edge locations within seconds. Each key can store up to 25 MB of data, and there is no limit on the number of keys per namespace.

4. Streaming Responses with SSE

Pages Functions support Server-Sent Events (SSE) for real-time streaming responses. This is essential for AI applications where language model outputs are generated token by token and should be displayed incrementally to the user.

To implement SSE streaming, the function creates a `TransformStream`, writes events to the writable side, and returns a `Response` with the readable side as the body. The `Content-Type` header must be set to `'text/event-stream'`.

Each SSE event follows the format: `'event: <type>\n\ndata: <json>\n\n'`. Common event types in AI applications include `'text_delta'` for incremental text, `'tool_called'` for function call notifications, `'citation'` for source references, and `'done'` for stream completion.

Error handling in streaming responses requires special attention. If an error occurs mid-stream, the function should emit an `'error'` event with details before closing the stream, allowing the client to display appropriate error messages.

5. Environment Variables and Secrets

EdgeOne Pages provides a secure environment variable system for managing API keys, database credentials, and other sensitive configuration. Variables are encrypted at rest and injected into the function runtime at execution time.

Environment variables are configured through the EdgeOne dashboard or CLI. They are accessible in Pages Functions via the standard `process.env` object or through the `context.env` property.

Best practices for environment variable management: (1) Never commit secrets to version control - use `.env.example` files with placeholder values. (2) Use descriptive naming conventions like `AI_GATEWAY_API_KEY` and `AI_GATEWAY_BASE_URL`. (3) Rotate credentials regularly and use the dashboard to update them without redeployment.

For local development, create a `.env` file in your project root with the same variable names. The EdgeOne CLI will automatically load these during local testing with the `'eo dev'` command.

6. Deployment and CI/CD

Deploying to EdgeOne Pages is straightforward. Connect your Git repository through the dashboard, and every push to your main branch triggers an automatic deployment. Preview deployments are created for pull requests, allowing team members to review changes before merging.

The build process supports popular frameworks including React, Vue, Next.js, and Astro. Custom build commands can be specified in the project settings. The platform automatically detects the framework and applies optimal build settings.

Deployment typically completes within 30-60 seconds. The platform performs automatic cache invalidation, ensuring users always receive the latest version. Rollbacks to previous deployments can be performed instantly through the dashboard.

For advanced workflows, the EdgeOne CLI supports programmatic deployments via 'eo deploy', enabling integration with any CI/CD system including GitHub Actions, GitLab CI, and Jenkins.